

HelixSkills

HelixSkills

A governed, constitution-backed skills system for CLI AI agents.

Summary

HelixSkills is a skills system for CLI AI agents that inherits the Helix Constitution as a submodule, so every universal governance rule applies unconditionally. It packages installable agent skills, MCP tool servers, Claude Code plugins, and reusable engines behind a registerable, documented catalog.

Short description

HelixSkills is a skills system for CLI AI agents. It embeds the Helix Constitution as a submodule so all universal rules apply, then ships registerable skills (action-prefix, media-validator, multitrack, session-sync, workable-item lifecycle, and more), two MCP tool servers, two Claude Code plugins, and reusable engines.

Long description

HelixSkills (repo `skills`, Apache-2.0) is a skills system for CLI AI agents, and it starts from a deliberate inversion of the usual order: governance comes first, capability second. It inherits the Helix Constitution as its constitution/ submodule, so every universal rule from `constitution/CLAUDE.md` and `constitution/Constitution.md` applies unconditionally — not as a convention an agent might honor, but as a rule set physically mounted into the project tree. An agent that adopts HelixSkills cannot opt out of the constitution; the rules travel with the code.

Where most “skill frameworks” trade in abstractions, HelixSkills ships a concrete, registerable inventory you can point at and install. Seven constitution skills install via `register.sh`: action-

prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, and workable-item-lifecycle — a spread rated from intermediate to advanced, covering everything from disciplined action naming to media validation to the full lifecycle of a unit of work. Additional draft skills (Android overview, Java/Kotlin language, Linux OS) are already indexed and staged, pending activation. Two MCP tool servers (media-validator, scheduled-work) surface those skills to agents over the Model Context Protocol, while two Claude Code plugins (helix, scheduled-work) drop the same capabilities straight into that agent runtime — one skill set, reaching agents through whichever surface they speak.

Underneath the catalog sit four depth-1 reusable engines — continuum (implemented), plus session_orchestrator, token_optimizer, and clickup_sync (in design) — the shared machinery that keeps skills from re-inventing the same plumbing. The token_optimizer alone declares an explicit dependency graph reaching into the vasic-digital ecosystem packages (TOON, Embeddings, VectorDB, Normalize, conversation) and HelixDevelopment's LLMProvider, so its cross-repo wiring is auditable rather than implicit. Around all of it runs disciplined documentation: a skills catalog, an auto-generated skill-graph index, per-repo detail pages, and a candid Gaps & Risks register that names what is not yet done. The whole system is mirrored across GitHub, GitLab, GitFlic, and GitVerse for resilience and regional access.

Why we built it

CLI AI agents need capabilities that are consistent, governed, and reusable — not ad-hoc scripts that each reinvent rules. HelixSkills was built to give agents a packaged, registerable skill set that is bound to a shared constitution, so behavior stays consistent and auditable across every agent and project that adopts it.

Why it's a game-changer

It makes agent capability portable and rule-compliant by *construction*, not by discipline. Every skill is a governed, versioned, installable unit backed by a constitution submodule — so the moment an agent registers a skill, it also inherits the canonical rule set, with no room to drift. That unlocks something that wasn't practical before: moving a capability from one agent or project to another and knowing it arrives already bound to the same governance, exposed through standard surfaces (MCP servers and Claude Code plugins) instead of a pile of bespoke glue scripts that each reinvent the rules.

What's innovative

- Constitution-as-submodule: universal governance rules are inherited, not copied — mounted into the tree so every consuming agent is bound to the same canonical rule set, with updates flowing through one source of truth instead of a dozen stale copies.
- Skills delivered as self-registering units (`register.sh`) and stitched into an auto-generated skill-graph index, so the catalog stays discoverable and never drifts out of sync with what's actually installed.
- Multi-surface exposure: the identical skill set reaches agents via MCP tool servers *and* Claude Code plugins — write once, speak to whichever runtime the agent uses.
- Reusable depth-1 engines (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`) shared across the ecosystem, each carrying explicit, auditable cross-repo dependency declarations rather than hidden coupling.

Biggest technical challenges & how we solved them

- **Keeping agent behavior consistent and rule-compliant across many skills and agents** — re-implementing governance per skill guarantees divergence over time. Solved by mounting the Helix Constitution as a submodule so the rules in `constitution/CLAUDE.md` and `constitution/Constitution.md` apply unconditionally and update from a single upstream, rather than being copied and left to rot.
- **Making a growing skill set installable and discoverable** — a catalog is worthless if nobody can find or install what's in it. Solved with per-skill `register.sh` registration that wires each skill in on install, plus an auto-generated INDEX skill graph and per-repo detail docs so discovery tracks reality automatically.
- **Reaching agents that speak different runtimes** — the same capability shouldn't be rebuilt for each host. Solved by packaging one skill set behind both MCP tool-server definitions (under `constitution/mcp/`) and Claude Code plugins (under `constitution/plugins/`), so a single implementation is exposed across surfaces.

Tech stack

- **Shell (primary language)** — chosen because install and registration tooling has to run anywhere an agent lives, with no runtime to bootstrap first; it powers `register.sh` and

install_upstreams, keeping the on-ramp dependency-free and portable.

- **Git submodules** — chosen to inherit governance without duplication: the Helix Constitution is mounted at constitution/ as a live reference, so rule updates propagate through one pointer instead of being copy-pasted and forgotten.
- **Model Context Protocol (MCP)** — chosen as the standard, runtime-agnostic tool interface for agents; two MCP servers (media-validator, scheduled-work) are defined under constitution/mcp/ to expose skills as callable tools.
- **Claude Code plugins** — chosen to drop skills natively into that agent runtime with zero glue; two plugins (helix, scheduled-work) ship under constitution/plugins/, mirroring the MCP surface for a different host.
- **Reusable engines (continuum, token_optimizer, session_orchestrator, clickup_sync)** — chosen to factor shared machinery out of individual skills for cross-project reuse; token_optimizer, for example, is wired to vasic-digital packages (T̄OON, Embeddings, VectorDB, Normalize, conversation) and HelixDevelopment's LLMProvider through declared dependencies rather than duplicated code.
- **Multi-host Git mirroring (GitHub, GitLab, GitFlic, GitVerse)** — chosen so a single host outage or regional block can't sever access; the same repository is kept live across four forges for resilience and reach.

Status & honesty notes

- **Status: beta.** The seven constitution skills, two MCP servers, and two plugins are shipped; the draft skills are indexed and pending activation, and three of the four depth-1 engines (session_orchestrator, token_optimizer, clickup_sync) are still in design.
- The README refers to the project as helix_skills; the canonical GitHub path is HelixDevelopment/skills. The tracked-findings count in the README is a self-reported figure.

Priority tier: Helix-primary.